

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
Факультет физико-математических и естественных наук

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 8

Студент: Николаева Ангелина

Борисовна

Студенческий билет:

1032253612

Группа: НКАбд-04-25

МОСКВА

2026 г

Содержание

1. Цель работы.....	4
2. Выполнение лабораторной работы	5
2.1. Реализация циклов в NASM.....	5
2.2. Обработка аргументов командной строки	9
2.3. Задание для самостоятельной работы	13
3. Выводы.....	16

Список иллюстраций

1.	Создание каталога	5
2.	Копирование программы из листинга	6
3.	Запуск программы	6
4.	Изменение программы	7
5.	Запуск измененной программы	8
6.	Добавление push и pop в цикл программы	8
7.	Запуск измененной программы	9
8.	Копирование программы из листинга	9
9.	Запуск второй программы.....	10
10.	Копирование программы из третьего листинга	11
11.	Запуск третьей программы.....	11
12.	Изменение третьей программы	12
13.	Запуск измененной третьей программы	12
14.	Написание программы для самостоятельной работы	13
15.	Запуск программы для самостоятельной работы.....	15

1. Цель работы

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки.

2. Выполнение лабораторной работы

2.1. Реализация циклов в NASM

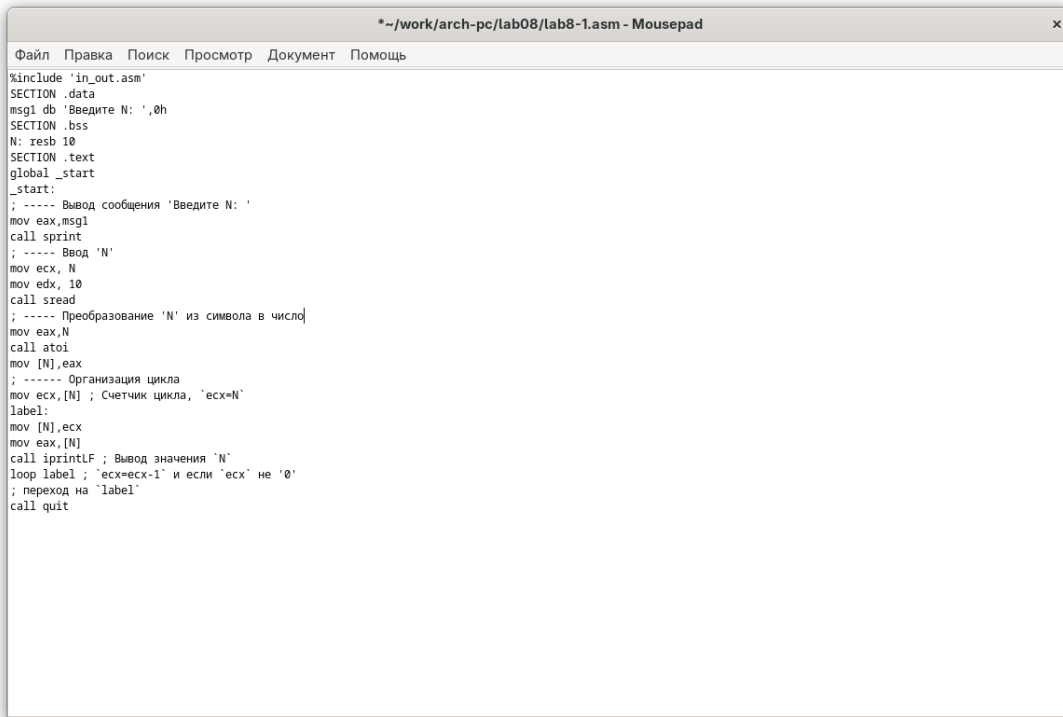
Создаю каталог для программ лабораторной работы №8 (рис. 4.1).



```
abnikilaeva@fedora:~/work/arch-pc/lab08
abnikilaeva@fedora:~/work/arch-pc/lab06$ cd
abnikilaeva@fedora:~$ mkdir ~/work/arch-pc/lab08
abnikilaeva@fedora:~$ cd ~/work/arch-pc/lab08
abnikilaeva@fedora:~/work/arch-pc/lab08$ touch lab8-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$
```

Рис. 4.1: Создание каталога

Копирую в созданный файл программу из листинга. (рис. 4.2).

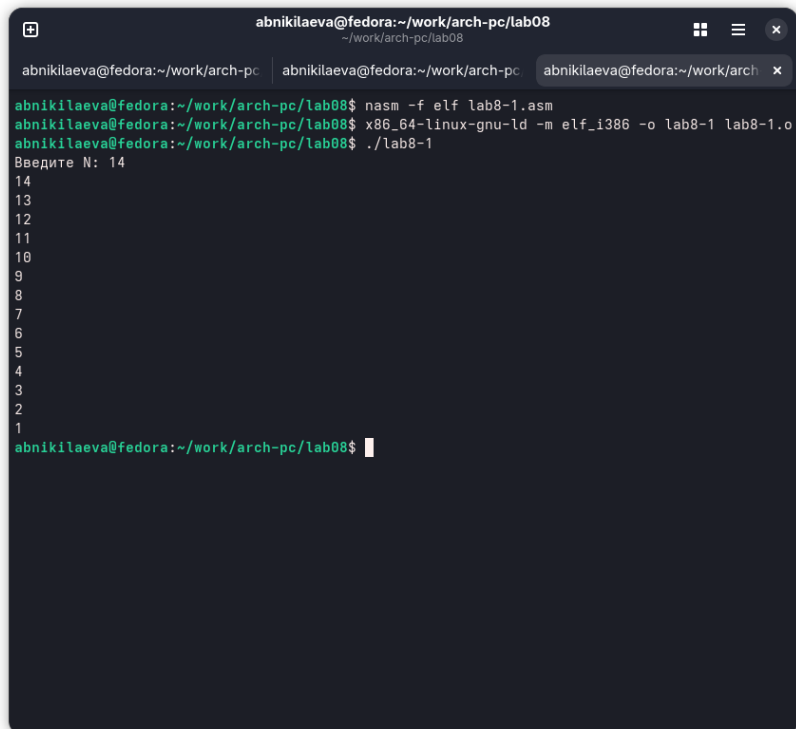


```
*~/work/arch-pc/lab08/lab8-1.asm - Mousepad
Файл  Правка  Поиск  Просмотр  Документ  Помощь

%include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ',0h
SECTION .bss
N: resb 10
SECTION .text
global _start
_start:
; ----- Вывод сообщения 'Введите N: '
mov eax,msg1
call sprint
; ----- Ввод 'N'
mov ecx, N
mov edx, 10
call sread
; ----- Преобразование 'N' из символа в число
mov eax,N
call atoi
mov [N],eax
; ----- Организация цикла
mov ecx,[N] ; Счетчик цикла, 'ecx=N'
label:
mov [N],ecx
mov eax,[N]
call iprintLF ; Вывод значения 'N'
loop label ; 'ecx=ecx-1' и если 'ecx' не '0'
; переход на 'label'
call quit
```

Рис. 4.2: Копирование программы из листинга

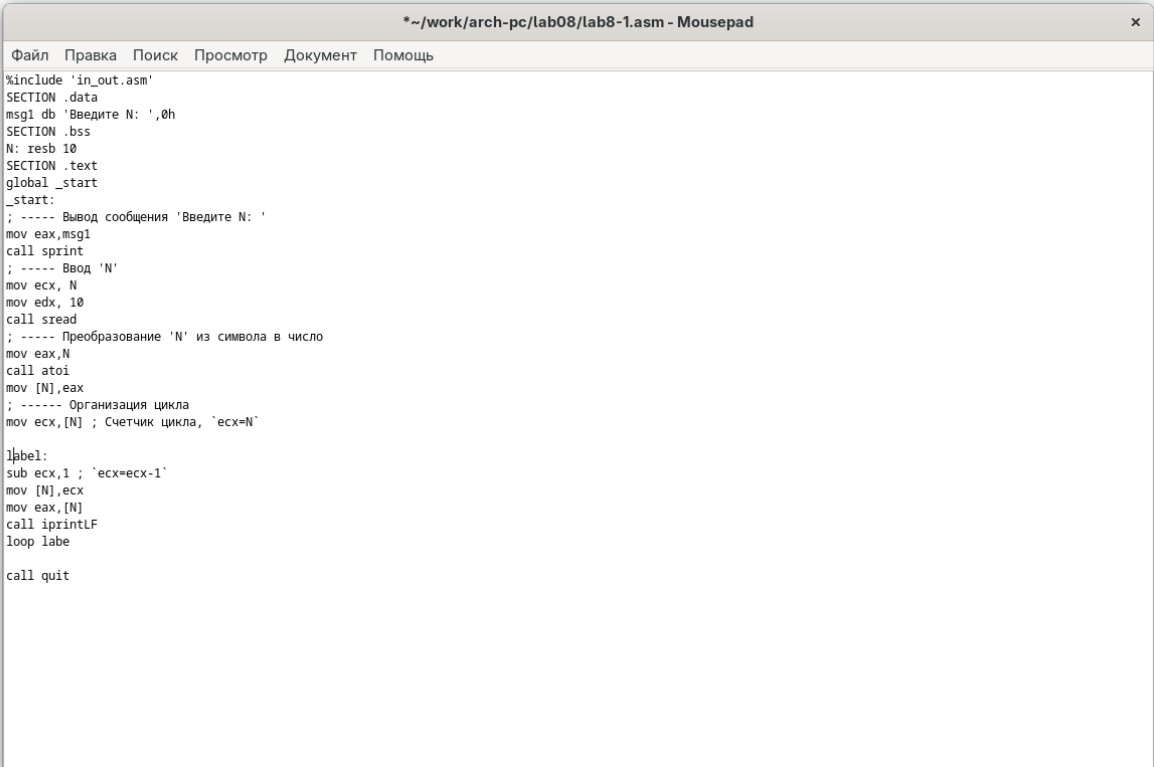
Запускаю программу, она показывает работу циклов в NASM (рис. 4.3).



```
abnikilaeva@fedora:~/work/arch-pc/lab08
~/work/arch-pc/lab08
abnikilaeva@fedora:~/work/arch-pc  abnikilaeva@fedora:~/work/arch-pc  abnikilaeva@fedora:~/work/arch-pc
abnikilaeva@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ x86_64-linux-gnu-ld -m elf_i386 -o lab8-1 lab8-1.o
abnikilaeva@fedora:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 14
14
13
12
11
10
9
8
7
6
5
4
3
2
1
abnikilaeva@fedora:~/work/arch-pc/lab08$
```

Рис. 4.3: Запуск программы

Заменяю программу изначальную так, что в теле цикла я изменяю значение регистра есх (рис. 4.4).



```
*~/work/arch-pc/lab08/lab8-1.asm - Mousepad
Файл  Правка  Поиск  Просмотр  Документ  Помощь

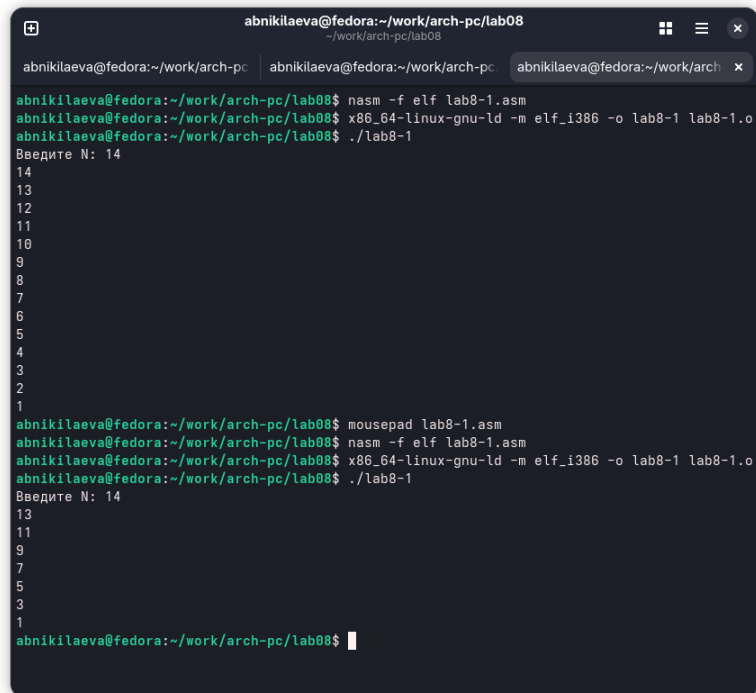
%include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ',0h
SECTION .bss
N: resb 10
SECTION .text
global _start
_start:
; ----- Вывод сообщения 'Введите N: '
mov eax,msg1
call sprint
; ----- Ввод 'N'
mov ecx, N
mov edx, 10
call sread
; ----- Преобразование 'N' из символа в число
mov eax,N
call atoi
mov [N],eax
; ----- Организация цикла
mov ecx,[N] ; Счетчик цикла, `ecx=N`

labe1:
sub ecx,1 ; `ecx=ecx-1`
mov [N],ecx
mov eax,[N]
call iprintLF
loop labe

call quit
```

Рис. 4.4: Изменение программы

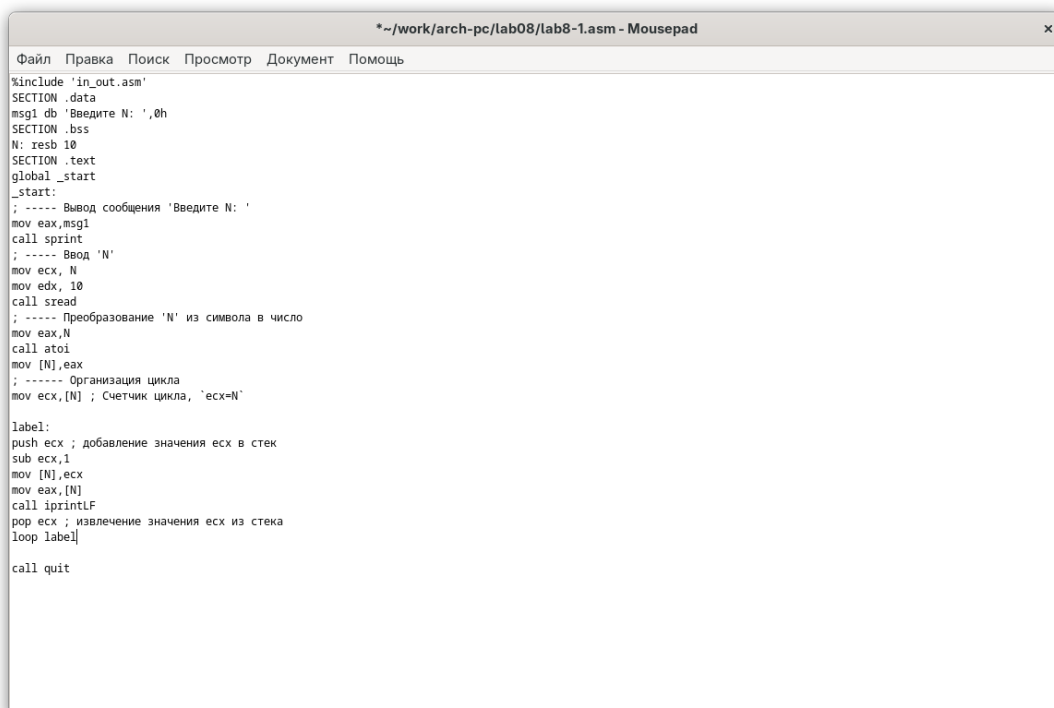
Из-за того, что теперь регистр есх на каждой итерации уменьшается на 2 значения, количество итераций уменьшается вдвое (рис. 4.5).



```
abnikilaeva@fedora:~/work/arch-pc/lab08
abnikilaeva@fedora:~/work/arch-pc
abnikilaeva@fedora:~/work/arch-pc
abnikilaeva@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ x86_64-linux-gnu-ld -m elf_i386 -o lab8-1 lab8-1.o
abnikilaeva@fedora:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 14
14
13
12
11
10
9
8
7
6
5
4
3
2
1
abnikilaeva@fedora:~/work/arch-pc/lab08$ mousepad lab8-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ x86_64-linux-gnu-ld -m elf_i386 -o lab8-1 lab8-1.o
abnikilaeva@fedora:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 14
13
11
9
7
5
3
1
abnikilaeva@fedora:~/work/arch-pc/lab08$
```

Рис. 4.5: Запуск измененной программы

Добавляю команды push и pop в программу (рис. 4.6).



```
*~/work/arch-pc/lab08/lab8-1.asm - Mousepad
Файл  Правка  Поиск  Просмотр  Документ  Помощь

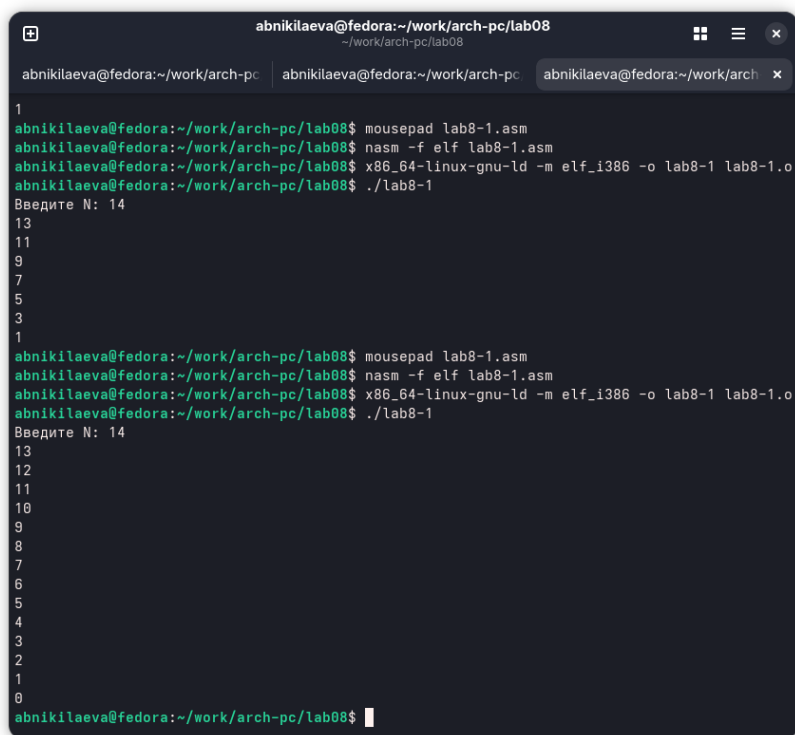
%include "in_out.asm"
SECTION .data
msg1 db 'Введите N: ',0h
SECTION .bss
N: resb 10
SECTION .text
global _start
_start:
; ----- Вывод сообщения 'Введите N: '
mov eax,msg1
call sprint
; ----- Ввод 'N'
mov ecx, N
mov edx, 10
call read
; ----- Преобразование 'N' из символа в число
mov eax,N
call atoi
mov [N],eax
; ----- Организация цикла
mov ecx,[N] ; Счетчик цикла, 'ecx=N'

label:
push ecx ; добавление значения ecx в стек
sub ecx,1
mov [N],ecx
mov eax,[N]
call iprintLF
pop ecx ; извлечение значения ecx из стека
loop label

call quit
```

Рис. 4.6: Добавление push и pop в цикл программы

Теперь количество итераций совпадает введенному N, но произошло смещение выводимых чисел на -1 (рис. 4.7).

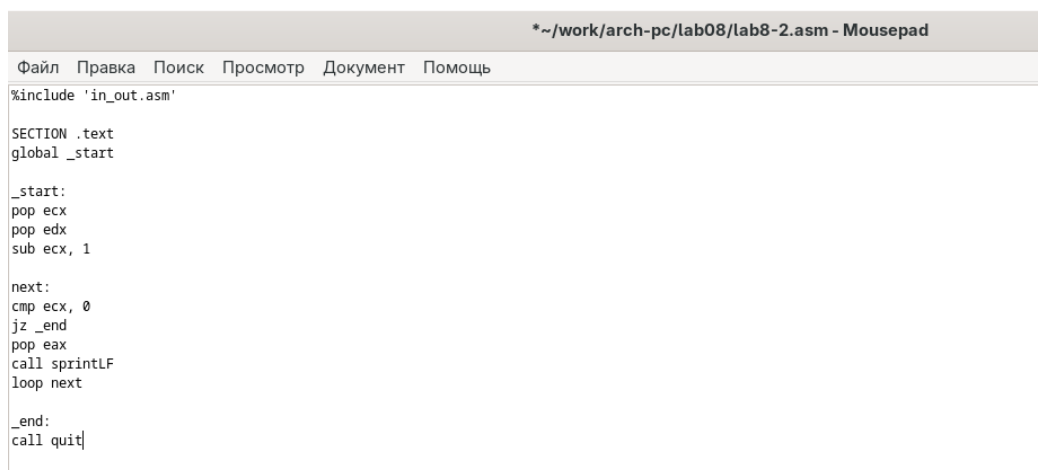


```
abnikilaeva@fedora:~/work/arch-pc/lab08
abnikilaeva@fedora:~/work/arch-pc labnikilaeva@fedora:~/work/arch
1
abnikilaeva@fedora:~/work/arch-pc/lab08$ mousepad lab8-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ x86_64-linux-gnu-ld -m elf_i386 -o lab8-1 lab8-1.o
abnikilaeva@fedora:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 14
13
11
9
7
5
3
1
abnikilaeva@fedora:~/work/arch-pc/lab08$ mousepad lab8-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ x86_64-linux-gnu-ld -m elf_i386 -o lab8-1 lab8-1.o
abnikilaeva@fedora:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 14
13
12
11
10
9
8
7
6
5
4
3
2
1
0
abnikilaeva@fedora:~/work/arch-pc/lab08$
```

Рис. 4.7: Запуск измененной программы

2.2.Обработка аргументов командной строки

Создаю новый файл для программы и копирую в него код из следующего листинга (рис. 4.8).



```
*~/work/arch-pc/lab08/lab8-2.asm - Mousepad
Файл Правка Поиск Просмотр Документ Помощь
%include 'in_out.asm'

SECTION .text
global _start

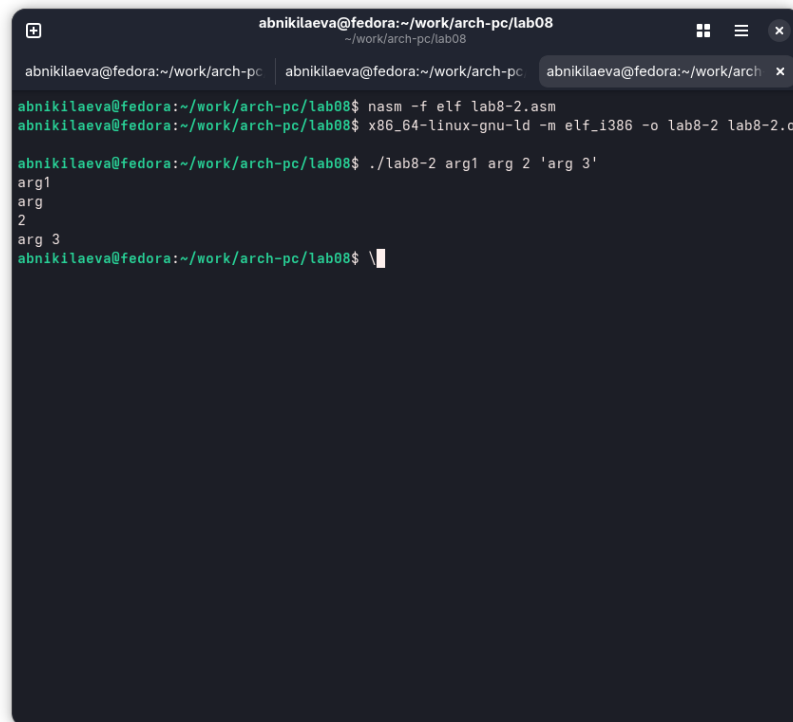
_start:
    pop ecx
    pop edx
    sub ecx, 1

next:
    cmp ecx, 0
    jz _end
    pop eax
    call sprintf
    loop next

_end:
    call quit|
```

Рис. 4.8: Копирование программы из листинга

Компилирую программу и запускаю, указав аргументы. Программой было обработано то же количество аргументов, что и было введено (рис. 4.9).



```
abnikilaeva@fedora:~/work/arch-pc/lab08
abnikilaeva@fedora:~/work/arch-pc  abnikilaeva@fedora:~/work/arch  x
abnikilaeva@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ x86_64-linux-gnu-ld -m elf_i386 -o lab8-2 lab8-2.o
abnikilaeva@fedora:~/work/arch-pc/lab08$ ./lab8-2 arg1 arg 2 'arg 3'
arg1
arg
2
arg 3
abnikilaeva@fedora:~/work/arch-pc/lab08$ \
```

Рис. 4.9: Запуск второй программы

Создаю новый файл для программы и копирую в него код из третьего листинга (рис. 4.10).

```
*~/work/arch-pc/lab08/lab8-3.asm - Mousepad
Файл Правка Поиск Просмотр Документ Помощь
%include 'in_out.asm'

SECTION .data
msg db "Результат: ", 0

SECTION .text
global _start

_start:
pop ecx
pop edx
sub ecx, 1
mov esi, 0

next: cmp ecx, 0
jz _end
pop eax
call atoi
add esi, eax

loop next

_end:
mov eax, msg
call sprint
mov eax, esi
call iprintf
call quit
```

Рис. 4.10: Копирование программы из третьего листинга

Компилирую программу и запускаю, указав в качестве аргументов некоторые числа, программа их складывает (рис. 4.11).

```
abnikilaeva@fedora:~/work/arch-pc/lab08$ touch lab8-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ mousepad lab8-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ x86_64-linux-gnu-ld -m elf_i386 -o lab8-3 lab8-3.o

abnikilaeva@fedora:~/work/arch-pc/lab08$ ./lab8-3 12 13 7 16 10 5
Результат: 63
abnikilaeva@fedora:~/work/arch-pc/lab08$ ./lab8-3 12 13 7 10 5
Результат: 47
abnikilaeva@fedora:~/work/arch-pc/lab08$
```

Рис. 4.11: Запуск третьей программы

Изменяю поведение программы так, чтобы указанные аргументы она умножала, а не складывала (рис. 4.12).

```
*~/work/arch-pc/lab08/lab8-3.asm - Mousepad
Файл  Правка  Поиск  Просмотр  Документ  Помощь
%include 'in_out.asm'

SECTION .data
msg db "Результат: ", 0

SECTION .text
global _start

_start:
pop ecx
pop edx
sub ecx, 1
mov esi, 1

next: cmp ecx, 0h
jz _end
pop eax
call atoi
mul esi, eax
mov esi, eax
loop next

_end:
mov eax, msg
call sprint
mov eax, esi
call iprintf
call quit
```

Рис. 4.12: Изменение третьей программы

Программа действительно теперь умножает данные на вход числа (рис. 4.13).

```
abnikilaeva@fedora:~/work/arch-pc/lab08
~/work/arch-pc/lab08

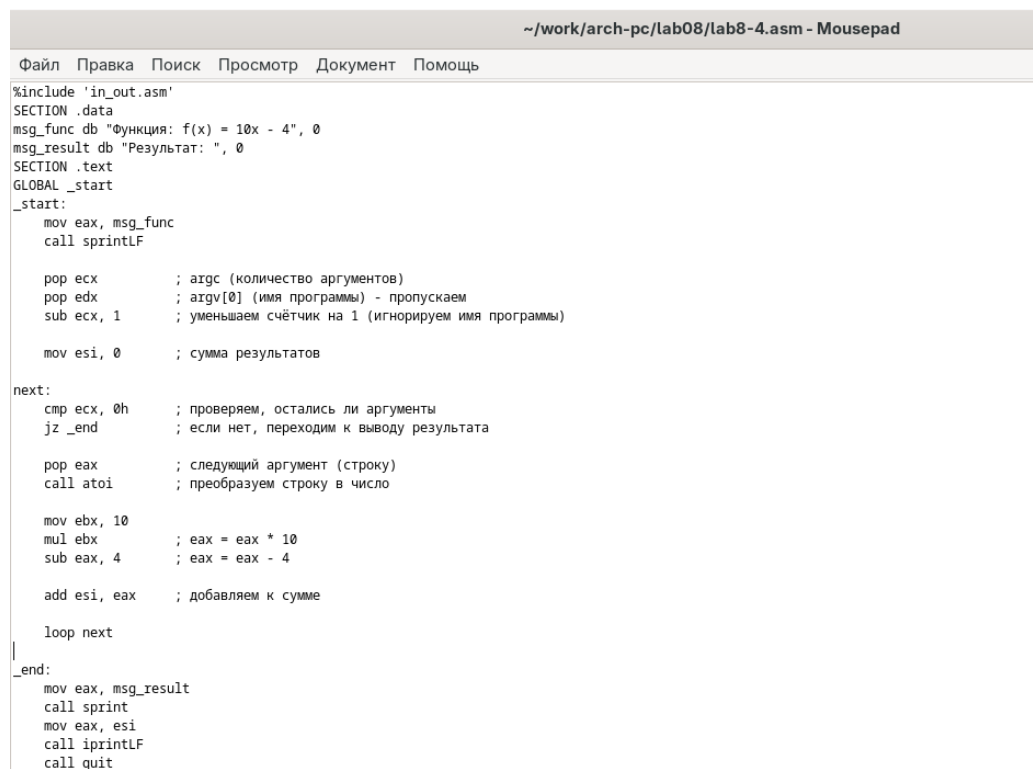
abnikilaeva@fedora:~/work/arch-pc/lab08$ mousepad lab8-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ x86_64-linux-gnu-ld -m elf_i386 -o lab8-3 lab8-3.o

abnikilaeva@fedora:~/work/arch-pc/lab08$ ./lab8-3 111 1 6
Результат: 666
abnikilaeva@fedora:~/work/arch-pc/lab08$
```

Рис. 4.13: Запуск измененной третьей программы

2.3. Задание для самостоятельной работы

Пишу программу, которая будет находить сумма значений для функции $f(x) = 10x - 4$ (рис. 4.14).



```
~/work/arch-pc/lab08/lab8-4.asm - Mousepad
Файл  Правка  Поиск  Просмотр  Документ  Помощь

%include 'in_out.asm'
SECTION .data
msg_func db "Функция: f(x) = 10x - 4", 0
msg_result db "Результат: ", 0
SECTION .text
GLOBAL _start
_start:
    mov eax, msg_func
    call sprintf

    pop ecx          ; argc (количество аргументов)
    pop edx          ; argv[0] (имя программы) - пропускаем
    sub ecx, 1       ; уменьшаем счётчик на 1 (игнорируем имя программы)

    mov esi, 0       ; сумма результатов

next:
    cmp ecx, 0h      ; проверяем, остались ли аргументы
    jz _end          ; если нет, переходим к выводу результата

    pop eax          ; следующий аргумент (строку)
    call atoi        ; преобразуем строку в число

    mov ebx, 10
    mul ebx          ; eax = eax * 10
    sub eax, 4       ; eax = eax - 4

    add esi, eax     ; добавляем к сумме

    loop next

_end:
    mov eax, msg_result
    call sprintf
    mov eax, esi
    call iprintf
    call quit
```

Рис. 4.14: Написание программы для самостоятельной работы

Код программы:

```
%include 'in_out.asm'

SECTION .data
msg_func db "Функция: f(x) = 10x - 4", 0
msg_result db "Результат: ", 0

SECTION .text
GLOBAL _start

_start:
    mov eax, msg_func
```

```
call sprintf
```

```
pop     pop
```

```
pop     edx
```

```
sub     ecx, 1
```

```
mov     esi, 0
```

```
next:
```

```
cmp     ecx, 0h
```

```
jz      _end
```

```
pop     eax
```

```
call    atoi
```

```
mov     ebx, 10
```

```
mul     ebx
```

```
sub     eax, 4
```

```
add     esi, eax
```

```
loop    next
```

```
_end:
```

```
mov     eax, msg_result
```

```
call    sprintf
```

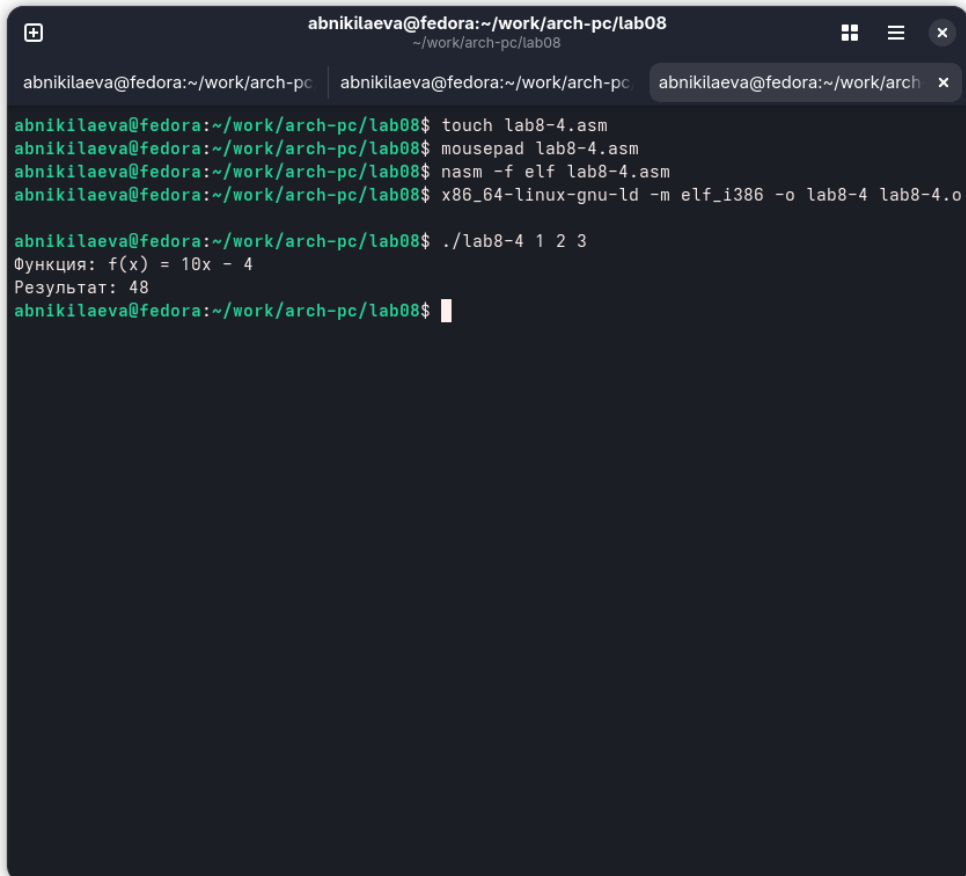
```
mov     eax,
```

```
esi call
```

```
iprintLF
```

```
call    quit
```

Проверяю работу программы, указав в качестве аргумента несколько чисел (рис. 4.15).



```
abnikilaeva@fedora:~/work/arch-pc/lab08
abnikilaeva@fedora:~/work/arch-pc | abnikilaeva@fedora:~/work/arch-pc | abnikilaeva@fedora:~/work/arch-pc
abnikilaeva@fedora:~/work/arch-pc/lab08$ touch lab8-4.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ mousepad lab8-4.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-4.asm
abnikilaeva@fedora:~/work/arch-pc/lab08$ x86_64-linux-gnu-ld -m elf_i386 -o lab8-4 lab8-4.o

abnikilaeva@fedora:~/work/arch-pc/lab08$ ./lab8-4 1 2 3
Функция:  $f(x) = 10x - 4$ 
Результат: 48
abnikilaeva@fedora:~/work/arch-pc/lab08$
```

Рис. 4.15: Запуск программы для самостоятельной работы

3. Выводы

В результате выполнения данной лабораторной работы я приобрел навыки написания программ с использованием циклов а также научился обрабатывать аргументы командной строки.